

Optimization Methods Assignment Report

PCB Drill Head Routing with Exact and Heuristic Optimization

Name: Lawrence Jerith
Student ID: MS26916026

12 October 2026

CONTENTS

List of Figures iii
List of Tables iii
Chapter One: Introduction 1
1.1 Dataset description 1
Chapter Two: Mathematical Formulation 3
2.1 Objective function 3
2.2 Constraints 3
Chapter Three: Exact Optimization on Small Subsets 5
3.1 Held–Karp dynamic programming 5
3.2 Implementation and limitations 6
Chapter Four: Exact Model for the Complete Board 7
4.1 Integer Linear Programming 7
4.2 Subtour elimination 7
4.3 Initial solution and stopping conditions 8
Chapter Five: Heuristic Optimization and Implementation 9
5.1 Nearest Neighbor algorithm 9
5.2 2-opt local search 9
5.3 Computational complexity 10
5.4 Implementation and validation 10
Chapter Six: Experimental Setup 11
Chapter Seven: Results Across Problem Sizes 13
7.1 Heuristic scalability 14
Chapter Eight: Comparison of the Best Solutions 16
8.1 Comparison of route distance 16
8.2 Comparison of execution time 17
8.3 Complete Drill-Head Routes 18
8.4 Subset Routes and Search Behavior 19
Chapter Nine: Discussion and Conclusion 21
9.1 Discussion 21
9.2 Conclusion 22
References 23
Appendix A: Complete Source Code 24

List of Figures

Figure 1. All 442 drilling locations in the pcb442 dataset……1
Figure 2. Execution time and mean heuristic optimality gap for the small subsets……13
Figure 3. Initial Nearest Neighbor and improved 2-opt distances for the complete board…….15
Figure 4. Complete drilling routes ILP model vs NN + 2-opt heuristic……..18
Figure 5. Comparison of Held–Karp and heuristic routes……..19
Figure 6. Improvement in route distance during the best heuristic……20

List of Tables

Table 1. Experimental settings……..11
Table 2. Comparison of exact and heuristic results on small subsets……13
Table 3. Heuristic performance on larger problems……..14
Table 4. Comparison of the best full-board results……16
Table 5. Comparison of full-board route characteristics……...19

Chapter One: Introduction

In PCB manufacturing, a drilling machine creates holes at specific locations on a circuit board. The drill head must move from one hole to another until all required holes have been drilled. Although the drilling locations are fixed, the order in which they are visited can be changed.

If the drilling locations are visited in an inefficient order, the drill head may travel unnecessary distances. By finding a better visiting order, the total movement can be reduced. This may help improve manufacturing efficiency, particularly when a board contains many drilling locations. This problem can be represented as a Travelling Salesman Problem (TSP). In the traditional TSP, a salesman must visit several cities exactly once and return to the starting city while travelling the shortest possible distance. In this application, the cities represent drilling locations, and the salesman represents the drill head.

The main objective of this study is to find a route that visits all drilling locations exactly once, returns to the starting location, and minimizes the total travel distance. Since the number of possible routes increases rapidly as more locations are added, finding the shortest route becomes computationally difficult.

1.1 Dataset description

The dataset used in this study is pcb442, obtained from the TSPLIB benchmark collection [1]. It contains 442 drilling locations represented by two-dimensional coordinates. The original dataset is stored in dataset/pcb442.tsp without modifying the coordinates. The dataset uses the EUC_2D distance format, meaning that the distance between two locations is calculated using Euclidean distance and rounded to the nearest integer according to TSPLIB rules. The coordinate units are not specified as millimeters, so the distances are reported in dataset units.

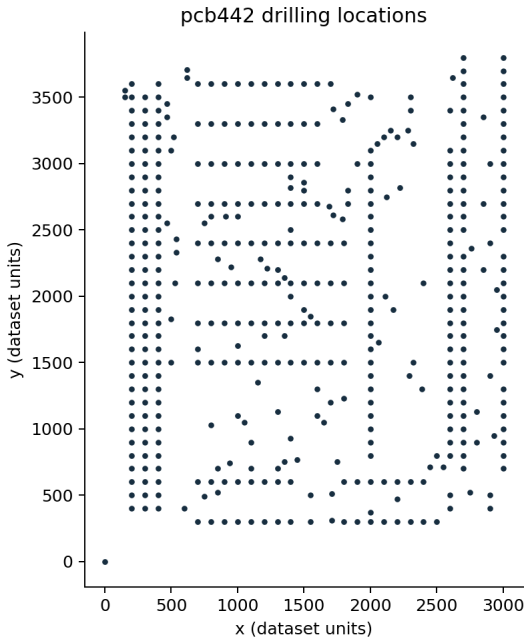


Figure 1. All 442 drilling locations in the pcb442 dataset.

Figure 1 shows the arrangement of the drilling locations across the board. Many locations are positioned along horizontal and vertical lines, while others are spread across different parts of the board. Finding a route that connects these locations efficiently is the main optimization problem.

The study assumes that a single drill head performs the operation, the travel distance is the same in both directions, and there are no obstacles between locations. It also assumes that every location must be visited once and that the drill head returns to its starting point. Tool changes, drilling order restrictions, acceleration, and other physical machine constraints are not included.

These assumptions allow the problem to be studied using the standard TSP formulation. The objective is to reduce the total route distance rather than directly calculate the time required by a physical drilling machine.

Chapter Two: Mathematical Formulation

The PCB drilling problem is modelled as a symmetric Travelling Salesman Problem. Let V represent the set of drilling locations and n represent the total number of locations.

$$V = \{1,2, \dots, n\}$$

For the complete PCB dataset, $n = 442$. Let E represent the possible connections between different drilling locations.

The distance between two locations i and j is calculated using their coordinates (x_i, y_i) and (x_j, y_j).

$$d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} + 0.5$$

The distance is rounded to the nearest integer using the TSPLIB EUC_2D rule. Each individual distance is rounded before calculating the total route distance. This is important because rounding only the final distance could produce a different result.

A binary decision variable x_{ij} is used to indicate whether the route includes the connection between locations i and j.

$$x_{ij} = \begin{cases} 1, & if\ the\ route\ uses\ edge\ between\ i\ and\ j \\ 0, & otherwise \end{cases}$$

2.1 Objective function

The objective is to minimize the total distance travelled by the drill head.

$$\min Z = \sum_{\{i,j\} \in E} d_{ij} x_{ij}$$

Here, Z represents the total route distance, d_{ij} is the distance between locations i and j, and x_{ij} indicates whether that connection is included in the route.

2.2 Constraints

Every drilling location must have exactly two connections in the final route: one connection for entering the location and another for leaving it.

However, this constraint alone does not guarantee a single complete route. It may allow several smaller closed routes, known as subtours, instead of one route covering all the locations.

To prevent this, a connectivity constraint is introduced.

$$\sum_{e \in \delta(S)} x_e \geq 2, \qquad \forall S \subset V, \quad 2 \leq |S| \leq n-2$$

Here, $\delta(S)$ represents the connections between a selected group of locations S and the remaining locations. This constraint ensures that the final route cannot be separated into independent cycles.

Finally, all decision variables must be binary.

$$x_{ij} \in \{0,1\}, \qquad \forall \{i,j\} \in E$$

Together, these constraints ensure that the solution forms one closed route that visits every drilling location exactly once.

Both optimization approaches in this study solve the same routing problem. The **exact approach** searches for an optimal solution using mathematical optimization, while the **heuristic approach** constructs and improves routes using simpler search rules.

The route distance is calculated by including the connection from the final drilling location back to the first. This ensures that all solutions are evaluated in the same way.

Chapter Three: Exact Optimization on Small Subsets

3.1 Held–Karp dynamic programming

The first exact optimization method implemented is the Held–Karp dynamic programming algorithm [4]. It is used to find the globally shortest route for small subsets of the drilling locations.

Dynamic programming works by dividing a large problem into smaller subproblems and storing their results. Instead of calculating the same partial routes repeatedly, the algorithm reuses previously calculated values.

For this implementation, location 0 is fixed as the starting point. Let S be a subset of locations excluding the starting point, and let j be the last visited location in that subset.

The function $C(S, j)$ represents the shortest distance required to travel from location 0, visit every location in S, and finish at location j.

The initial condition is:

$$C(\{j\}, j) = d_{0j}$$

This represents the distance from the starting point directly to location j.

For larger subsets, the shortest distance is calculated using:

$$C(S, j) = \min_{k \in Sj} \left[C(Sj\ , k) + d_{kj} \right]$$

This equation considers every possible location k that could be visited immediately before j. The algorithm chooses the option with the smallest total distance.

Once all locations have been considered, the shortest complete route is calculated by returning to the starting location.

$$L^* = \min_{j \in V0} \left[C(V0\ , j) + d_{j0} \right]$$

Here, L^* represents the globally optimal route distance for the selected subset.

The method guarantees the shortest route because it considers all possible final locations and uses the shortest available path for every intermediate state.

3.2 Implementation and limitations

The Held–Karp algorithm is implemented in *scripts/exact_method.py*. A bit mask is used to represent the locations included in each subset. NumPy arrays store the shortest distances and the previous locations needed to reconstruct the final route.

The algorithm has the following computational complexity:

$$\text{Time complexity} = O(n^2 2^n)$$

$$\text{Space complexity} = O(n2^n)$$

These expressions show that the computational requirements increase exponentially with the number of locations. As a result, Held–Karp is practical for small problems but not suitable for all 442 drilling locations.

For example, at 18 locations, the two main arrays require approximately 19.1 MiB of memory. This is the calculated array allocation rather than the total memory used by the program.

In this study, Held–Karp is tested on subsets containing 8, 10, 12, 14, 16, and 18 locations. Each subset is solved three times to measure execution time. The implementation also limits the input size to between 3 and 20 locations to prevent excessive memory and processing requirements.

The main purpose of Held–Karp is to provide proven optimal solutions for smaller problems. These solutions are then used to evaluate the accuracy of the heuristic approach.

Chapter Four: Exact Model for the Complete Board

4.1 Integer Linear Programming

Since Held–Karp cannot practically solve the complete 442-location problem, an Integer Linear Programming (ILP) model is used for the full dataset.

The ILP model follows the mathematical formulation described in Section 2. It uses binary variables to represent connections between drilling locations and attempts to find the combination of connections that produces the shortest valid route.

The implementation uses *PySCIPOpt* 6.0.0 with the *SCIP* 10.0.0 optimization solver [6]. For 442 locations, the number of possible undirected connections is:

$$|E| = \frac{n(n-1)}{2}$$

$$|E| = \frac{442(441)}{2} = 97{,}461$$

Therefore, the model contains 97,461 binary decision variables and 442-degree constraints. All possible connections are included in the model.

SCIP uses a branch-and-bound-based optimization process to search for good solutions while calculating bounds on the best possible result.

4.2 Subtour elimination

A major challenge in the TSP formulation is preventing the solver from producing several disconnected cycles.

For example, the solver could create one closed route through some locations and another closed route through the remaining locations. Although every location would have two connections, the result would not represent a valid single drill-head route.

To prevent this, subtour elimination constraints are added.

$$\sum_{e \in E(S)} x_e \leq |S| - 1, \qquad \forall S \subset V, \quad 2 \leq |S| \leq n-1$$

Here, $E(S)$ represents the connections between locations within subset S. This constraint prevents a smaller subset from forming its own closed route.

Adding every possible subtour constraint at the beginning would be computationally expensive. Instead, the implementation uses a custom constraint handler that checks the solutions produced by SCIP. When disconnected cycles are found, additional constraints are introduced to eliminate them.

This allows the solver to work with a smaller initial model and add connectivity constraints when they are required.

4.3 Initial solution and stopping conditions

Before starting the ILP optimization, an initial valid route is generated using the Nearest Neighbor algorithm followed by 2-opt local search. This route is provided to SCIP as a warm start.

A warm start gives the solver a feasible solution at the beginning of the optimization process. The solver can then attempt to find a better solution.

The ILP experiment uses one processing thread, a feasibility tolerance of 10^{-6}, and a relative optimality-gap target of zero. The solver is given a maximum solving time of 300 seconds. The total recorded execution time also includes model preparation, warm-start construction, and solution extraction.

The solver completed the experiment with the following results:

- Best route distance: 50,778
- Solver lower bound: 50,737
- Relative solver gap: 0.0808%
- Subtour constraints added: 193
- Branch-and-bound nodes processed: 586
- Solver status: Time limit reached

The route distance of 50,778 matches the published optimal distance for pcb442 [3]. However, the solver's lower bound remained at 50,737 when the time limit was reached. This means that the solver did not independently prove that its final solution was globally optimal. The result is therefore reported as a feasible route matching the published optimum, rather than a solution independently proven optimal by this solver run.

Chapter Five: Heuristic Optimization and Implementation

5.1 Nearest Neighbor algorithm

The second optimization approach combines the Nearest Neighbor (NN) algorithm with 2-opt local search. This approach is selected because it is simple to implement and can find good routes with relatively low computational cost.

The Nearest Neighbor algorithm begins at a selected drilling location and repeatedly visits the closest location that has not yet been visited. This process continues until all locations have been included in the route. The route is then completed by returning to the starting location.

Although this method is fast, it does not always produce the shortest route. A decision that appears good at the beginning may lead to unnecessary travel later. For this reason, the initial route is improved using 2-opt local search.

5.2 2-opt local search

The 2-opt algorithm improves an existing route by replacing two connections with two different connections. It selects two non-adjacent edges, removes them, and reconnects the route by reversing the section between them.

Suppose the selected edges connect locations (a, b) and (c, d). The algorithm replaces them with connections (a, c) and (b, d). The change in total route distance is calculated as:

$$\Delta L = d_{ac} + d_{bd} - d_{ab} - d_{cd}$$

A negative value of ΔL means that the new connections produce a shorter route. A positive value means that the change would increase the total distance.

The implementation evaluates the possible 2-opt exchanges and selects the one that gives the greatest reduction in distance. After applying the improvement, the process is repeated. The algorithm stops when no further 2-opt exchange can reduce the route distance. At this point, the route is considered a local optimum.

However, a local optimum is not necessarily the shortest possible route. It only means that no improvement can be found using the 2-opt changes being considered. A different starting route or a more advanced optimization method may still produce a shorter solution.

5.3 Computational complexity

The Nearest Neighbor algorithm has a time complexity of:

$$O(n^2)$$

A complete 2-opt search also requires approximately $O(n^2)$ operations for each improvement step.

If the algorithm accepts I improving moves, the overall scanning cost is approximately:

$$O\big((I+1)n^2\big)$$

The number of improving moves depends on the initial route and the arrangement of locations. Therefore, the total execution time can vary between different runs.

In this study, the heuristic is executed using ten different random seeds. Each seed determines the starting location for the Nearest Neighbor algorithm. The best route among the ten runs is selected for comparison.

5.4 Implementation and validation

The algorithms are implemented in Python, with NumPy used for numerical calculations and Matplotlib used to visualize the results.

The project separates the main tasks into several Python files. The exact_method.py file contains the Held–Karp algorithm, while *heuristic_method.py* implements Nearest Neighbor, 2-opt local search, and route-distance calculations. The *ilp_method.py* file contains the integer programming model, and *compare_models.py* performs the full-board comparison. The *main.py* file manages the original experiments, while *verify.py* contains tests to check that the algorithms produce valid results.

Several validation checks are included to confirm the correctness of the implementation. These checks verify that every drilling location is visited exactly once, the route returns to its starting point, and the calculated distance agrees with the stored result. The Held–Karp implementation is also compared with brute-force solutions for small test problems. The ILP implementation is checked against Held–Karp on four examples. An additional six-location test confirms that the ILP model correctly prevents disconnected subtours.

All implemented validation checks passed.

Chapter Six: Experimental Setup

The experiments are designed to compare the two optimization approaches in terms of solution quality and computational time. The same distance calculation is used for both approaches to ensure that their results can be compared fairly.

The original experiment considers different problem sizes to examine how the algorithms behave as the number of drilling locations increases. Small subsets containing 8, 10, 12, 14, 16, and 18 locations are used to compare Held–Karp against the heuristic. Larger problems containing 50, 100, 200, and 442 locations are used to evaluate the heuristic.

The subsets are selected using NumPy with a fixed random seed of 2026. Locations are first randomly ordered, and progressively larger subsets are selected from this ordering. This ensures that both algorithms solve the same small problems, although the subsets are related because they share locations.

A separate experiment is conducted to compare the ILP model and heuristic on the complete 442-location dataset. The ILP model is executed once, while the heuristic is executed ten times using different starting seeds.

Table 1. Experimental settings

Parameter	**Setting**
Dataset	TSPLIB pcb442
Number of drilling locations	442
Small problem sizes	8, 10, 12, 14, 16, 18
Larger problem sizes	50, 100, 200, 442
Held–Karp repetitions	3 per small subset
Heuristic repetitions	10 runs per problem size
Full-board ILP runs	1
ILP solving time limit	300 seconds
ILP processing threads	1
ILP initial solution	Nearest Neighbour + 2-opt, seed 0
Programming language	Python 3.12.14
Operating system	macOS 26.1, ARM64
Libraries	NumPy 2.2.6, Matplotlib 3.10.7, PySCIPOpt 6.0.0, SCIP 10.0.0

The original experiment contains 118 algorithm runs, and the dedicated full-board comparison contains another 11 runs. The results are stored in separate files to keep the measurements from the two experiments identifiable.

Execution time is measured using elapsed software runtime. This includes the time required by the optimization algorithms, along with their relevant setup operations. Dataset loading, distance-matrix generation, figure creation, and external validation are excluded from the measured times.

The quality of a solution is evaluated by comparing its route distance with a known optimal distance whenever one is available.

The percentage gap is calculated as:

$$\text{Gap (\%)} = \frac{L - L^*}{L^*} \times 100$$

Here, L is the distance of the solution being evaluated, and L^* is the known optimal distance.

A gap of 0% indicates that the route matches the known optimum. A higher percentage indicates that the route is longer than the optimum. For problem sizes of 50, 100, and 200 locations, a proven optimal distance is not available from these experiments. Therefore, an optimality gap is not reported for those subsets.

Chapter Seven: Results Across Problem Sizes

The first experiment compares Held–Karp dynamic programming and the Nearest Neighbor with 2-opt heuristic on small subsets of the dataset.

Held–Karp is used to calculate the exact shortest route for each subset. The heuristic is executed ten times, and its best result is compared with the exact solution.

Table 2. Comparison of exact and heuristic results on small subsets

Locations	Held–Karp distance	Heuristic distance	Difference	Gap (%)	Held–Karp mean time (s)	Heuristic total time (s)
8	9,282	9,282	0	0.00	0.000427	0.000917
10	10,597	10,794	197	1.86	0.002887	0.000915
12	11,205	11,269	64	0.57	0.016327	0.000992
14	11,351	11,351	0	0.00	0.087687	0.001090
16	12,068	12,068	0	0.00	0.483784	0.001216
18	12,073	12,073	0	0.00	2.325445	0.001452

The results show that the heuristic can produce optimal solutions for several of the smaller problems. It matched the Held–Karp solution for subsets containing 8, 14, 16, and 18 locations.

However, it did not find the optimal solution for every subset. For 10 locations, Held–Karp found a route distance of 10,597, while the best heuristic result was 10,794. This gives a difference of 197 units, or 1.86%. For 12 locations, the heuristic result was 64 units longer than the optimum, corresponding to a gap of 0.57%.

These results show that the heuristic can produce good solutions but running it, several times does not guarantee that it will find the global optimum.

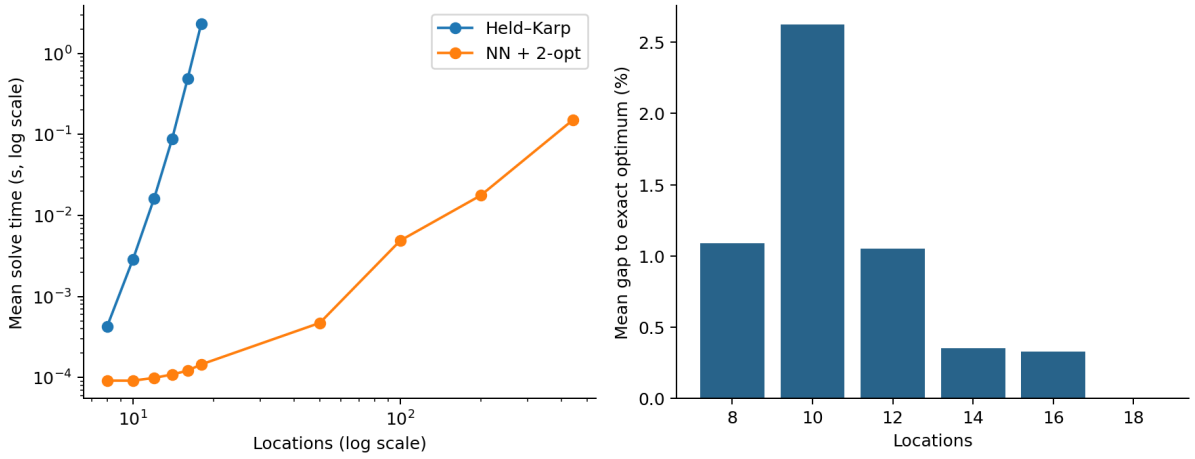


Figure 2. Execution time and mean heuristic optimality gap for the small subsets.

Figure 2 compares how the execution times change with problem size and shows the average heuristic gap for each subset. The Held–Karp execution time increases rapidly as more locations are included, while the heuristic remains much faster.

For example, Held–Karp required an average of 2.325 seconds for 18 locations. In comparison, a single heuristic run for the same subset took approximately 0.000145 seconds on average. The Held–Karp execution time increased by approximately 5,445 times between the 8-location and 18-location problems. This agrees with the expected exponential growth of the dynamic programming algorithm.

An interesting observation is that the heuristic found the exact optimum for the 18-location subset, even though it did not do so for the smaller 10-location and 12-location subsets. This suggests that the quality of a heuristic solution depends not only on the number of locations but also on how those locations are arranged.

7.1 Heuristic scalability

The second part of the original experiment evaluates the heuristic on larger problems containing 50, 100, 200, and 442 drilling locations.

The algorithm is executed ten times for each problem size. The best, average, and worst distances are recorded along with the standard deviation and mean execution time.

Table 3. Heuristic performance on larger problems

Locations	Best distance	Mean distance	Worst distance	Standard deviation	Mean time (s)
50	17,383	17,421.2	17,586	70.0	0.000472
100	23,977	24,598.5	25,259	482.5	0.004902
200	33,446	34,007.3	34,685	420.2	0.017689
442	52,205	52,824.4	53,574	424.4	0.150590

The results show that the heuristic can solve larger problems within a short time. Even for the complete dataset of 442 locations, the average execution time was only 0.150590 seconds in the original experiment.

For the complete board, the best result was obtained using seed 6, with a route distance of 52,205. The mean distance across the ten runs was 52,824.4, while the worst result was 53,574. The best heuristic solution was 2.81% longer than the published optimal distance of 50,778.

The mean result was approximately 4.03% longer than the published optimum.

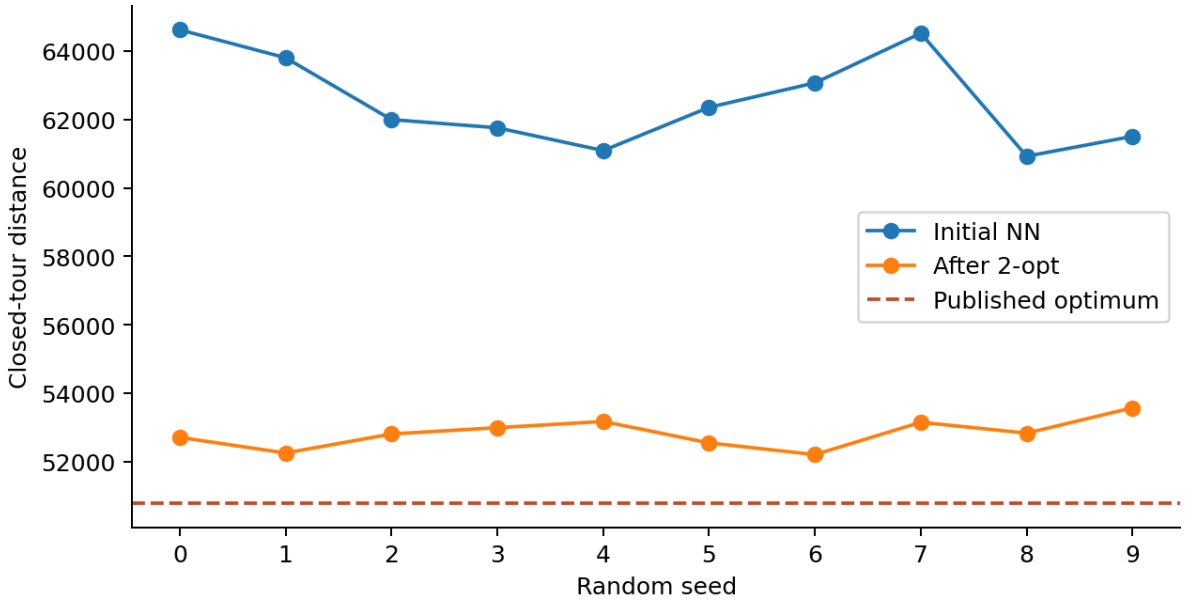


Figure 3. Initial Nearest Neighbor and improved 2-opt distances for the complete board.

Figure 3 shows the improvement achieved by applying 2-opt local search to the initial Nearest Neighbor routes. The average starting distance was 62,565.4, which was reduced to an average final distance of 52,824.4.

This represents an average reduction of approximately 15.53% in route distance. The result demonstrates that 2-opt local search can significantly improve the initial routes produced by Nearest Neighbor.

The execution times also increase as the number of locations grows. However, the heuristic remains practical for all problem sizes tested in this study. These results apply to the current implementation and dataset, and do not necessarily represent its performance on every possible routing problem.

Chapter Eight: Comparison of the Best Solutions

The main comparison focuses on the complete pcb442 dataset containing 442 drilling locations. Both approaches use the same distance matrix and aim to minimize the total distance of a closed route that visits every location exactly once.

The ILP approach uses its best feasible solution from one SCIP optimization run. The heuristic approach uses the best solution obtained from ten runs with different starting seeds. The comparison considers three main factors: the final route distance, the execution time, and the level of confidence in the solution's optimality.

Table 4. Comparison of the best full-board results

Measure	**ILP model**	**NN + 2-opt heuristic**
Best route distance	50,778	52,205
Solution status	Feasible; no independent solver proof	2-opt local optimum
Gap to published optimum	0.0000%	2.8103%
Solver lower bound	50,737	—
Solver relative gap	0.0808%	—
Number of runs	1	10
Selected run time (s)	301.532766	0.235001
Total execution time (s)	301.532766	2.514426
Mean execution time (s)	301.532766	0.251443
Locations visited	442	442
Closed route	Yes	Yes
Initial solution	NN + 2-opt, seed 0	NN, seeds 0–9
Best heuristic seed	—	6

8.1 Comparison of route distance

The ILP model produced a route distance of 50,778, which matches the published optimal distance for the pcb442 benchmark [3]. The best heuristic route had 52,205.

The difference between the two solutions is:

$$\Delta L = 52{,}205 - 50{,}778$$

$$\Delta L = 1{,}427$$

Therefore, the heuristic route is 1,427 distance units longer than the ILP route.
The percentage difference relative to the published optimum is:

$$\text{Gap} = \frac{52{,}205 - 50{,}778}{50{,}778} \times 100$$

$$\text{Gap} = 2.81\%$$

This shows that the heuristic produced a reasonably close solution, despite using a much simpler optimization process.

When the ILP result is compared directly against the heuristic distance, the reduction is:

$$\text{Reduction} = \frac{52{,}205 - 50{,}778}{52{,}205} \times 100$$

$$\text{Reduction} = 2.73\%$$

Therefore, the ILP route is approximately 2.73% shorter than the best heuristic route.

The ILP solver initially received a warm-start route with 52,709. During optimization, this distance was reduced to 50,778, giving an improvement of 1,931 units over its initial solution. Although the ILP result matches the published optimum, the solver did not independently confirm optimality before reaching the time limit. Its final lower bound was 50,737, leaving a relative solver gap of 0.0808%.

It is important to distinguish between matching a published optimal result and obtaining an independent optimality proof. In this experiment, the ILP solution matches the benchmark optimum, but the recorded solver run does not provide its own complete proof.

8.2 Comparison of execution time

The most noticeable difference between the two approaches is their execution time.
The ILP model required approximately 301.533 seconds to complete its recorded run. In comparison, the ten heuristic runs required only 2.514 seconds in total.

The ratio between these execution times is:

$$\text{Time ratio} = \frac{301.532766}{2.514426}$$

$$\text{Time ratio} \approx 119.9$$

This means that the ILP approach took approximately 120 times longer than all ten heuristic runs combined.

The ILP model requires more time because it searches for improved solutions while also attempting to establish mathematical bounds on the best possible solution. The heuristic does not perform this additional optimality verification. It only constructs routes and improves them until no further 2-opt improvement is possible.

However, the comparison has some limitations. The ILP model was executed only once, while the heuristic was executed ten times. The methods were also not given equal execution-time budgets. In addition, the ILP model used a heuristic solution as its starting point.

Despite these limitations, the recorded results show a clear difference between the approaches. The heuristic is suitable when a good solution is required quickly, while the ILP model is more useful when reducing the route distance and obtaining stronger optimality evidence are important.

8.3 Complete Drill-Head Routes

The complete routes obtained from the ILP model and the best heuristic run are visualized to compare how they move between the 442 drilling locations.

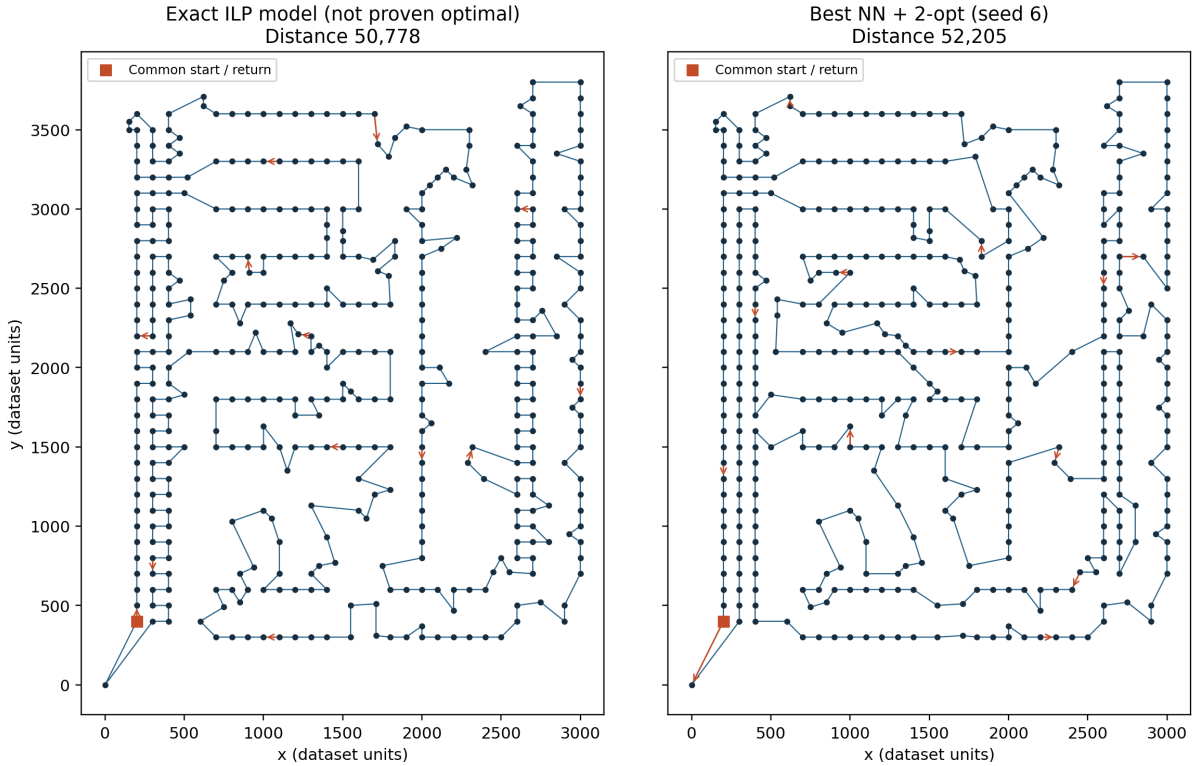


Figure 4. Complete drilling routes obtained using the ILP model (left) and NN + 2-opt heuristic (right).

Both plots display every drilling location and the connections between them. The same coordinate axes are used so that the routes can be compared directly. Location 1 is used as the displayed starting point, and the final connection back to the starting point is included.

The plots show that both methods generally connect nearby drilling locations. However, their visiting orders differ in several areas, producing different total route distances.

To examine these differences further, several characteristics of the routes are compared.

Table 5. Comparison of full-board route characteristics

Measure	ILP model	Best heuristic
Number of locations	442	442
Number of edges	442	442
Total route distance	50,778	52,205
Average edge distance	114.882	118.111
Longest edge distance	500	500
Edges shared between routes	320 of 442	320 of 442

Both routes contain 442 connections and visit all 442 locations exactly once. The average distance of an edge in the ILP route is 114.882, compared with 118.111 for the heuristic. The longest edge in both routes has 500 units. This indicates that the difference in total route distance does not come from a single unusually long connection. Instead, it results from the combined effect of many connection choices throughout the route.

The two routes share 320 of their 442 undirected edges. This means that a large portion of their connections is the same, but they use different connections in other areas.These differences explain why the ILP route is shorter, even though the overall appearance of the two routes is similar.

Since the problem is modelled as a symmetric closed TSP, the starting point can be changed without affecting the total route distance. Similarly, travelling along the same route in the opposite direction gives the same distance.

The route visualizations are useful for understanding the solutions, but they are not sufficient to verify correctness. The saved routes are also checked programmatically to confirm that every location is visited once, and the total distance is calculated correctly.

8.4 Subset Routes and Search Behavior

The difference between exact and heuristic optimization can be understood more clearly by examining the smaller 10-location problem.

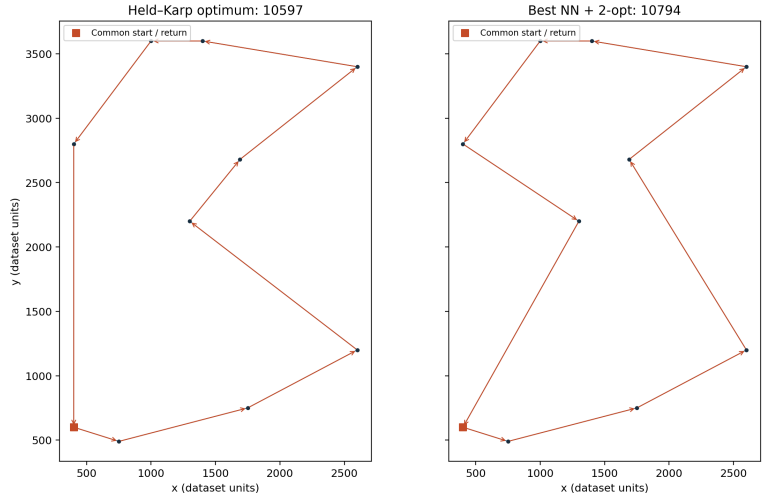


Figure 5. Comparison of Held–Karp and heuristic routes for the 10-location subset.

For this subset, Held–Karp produced an optimal route distance of 10,597, while the best heuristic route had 10,794. Both routes visit exactly the same locations, but they connect them in a different order. The heuristic solution is valid, yet it is 197 units longer than the exact solution.

This example demonstrates an important limitation of local search. Even after the 2-opt algorithm reaches a point where no further improvement is possible, its route may still be longer than the global optimum.

The behaviour of 2-opt can also be observed by plotting the route distance after each accepted improvement.

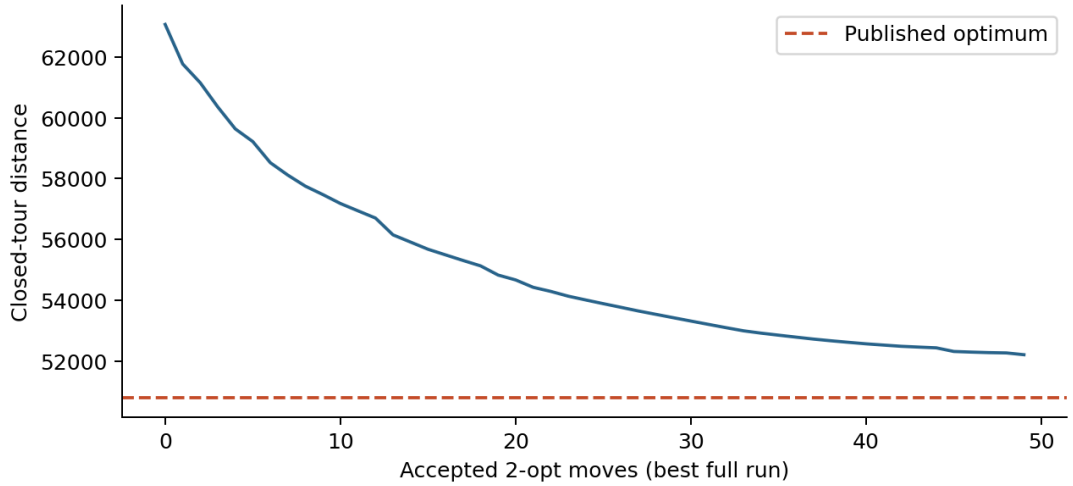


Figure 6. Improvement in route distance during the best heuristic run on the complete board.

For the complete dataset, the best heuristic run was obtained using seed 6. Its initial Nearest Neighbor route had 63,071. The 2-opt algorithm then performed 49 improving moves, gradually reducing the distance until it reached 52,205.

Every accepted move decreased the total distance, which explains the downward trend shown in Figure 6. The algorithm stopped when no further improvement was possible using a single 2-opt exchange.

However, this does not mean that the final solution is globally optimal. Other optimization methods, such as 3-opt or more advanced local search techniques, may be able to improve the route further.

Chapter Nine: Discussion and Conclusion

9.1 Discussion

The results demonstrate the main differences between exact and heuristic optimization methods when applied to the PCB drill-head routing problem.

The Held–Karp algorithm successfully calculated globally optimal routes for the smaller problem sizes. Its main advantage is that it guarantees an optimal solution. However, its execution time and memory requirements increase exponentially, making it unsuitable for the complete 442-location dataset.

The ILP model makes it possible to apply an exact mathematical formulation to the complete board. It produced a route distance of 50,778, matching the published optimal result. However, the solver reached its time limit before its lower bound matched the solution distance. Therefore, the result cannot be reported as independently proven optimal by the recorded solver run.

The Nearest Neighbor with 2-opt heuristic performed well across the tested problem sizes. It found optimal solutions for several small subsets and produced a full-board route only 2.81% longer than the published optimum.

The results also show that 2-opt provides a considerable improvement over Nearest Neighbor alone. For the complete board, the average route distance decreased by 15.53% after applying local search.

The most important advantage of the heuristic is its speed. While the ILP approach required approximately 301.533 seconds, the ten full-board heuristic runs required only 2.514 seconds in total. This makes the heuristic a useful choice for applications where a route must be calculated quickly and a small difference from the optimal distance is acceptable.

However, the study also has several limitations. Only one set of related subsets was used, and the heuristic was tested using ten starting seeds per problem size. The ILP model was tested only once on the complete dataset. As a result, the experiments are not sufficient to make general statistical conclusions about the performance of these algorithms on other datasets.

The execution-time comparison is also affected by differences in implementation and the use of a heuristic warm start in the ILP model. In addition, the recorded times represent software execution rather than the actual time required by a physical drilling machine.

The optimization objective considers only travel distance. Other practical factors, such as drill-head acceleration, machine movement speed, tool changes, and drilling time, are not included. Therefore, the measured improvements in route distance cannot be directly interpreted as equivalent reductions in manufacturing time.

9.2 Conclusion

This study implemented and compared exact and heuristic optimization approaches for PCB drill-head routing using the TSPLIB pcb442 dataset.

Held–Karp dynamic programming was used to calculate optimal routes for small subsets, while an ILP model was developed for the complete 442-location problem. A heuristic approach combining Nearest Neighbor and 2-opt local search was also implemented and evaluated. The ILP model produced a full-board route distance of 50,778, matching the published optimum, although the solver did not independently prove optimality within its time limit. The best heuristic solution had a distance of 52,205, which was 2.81% longer than the published optimum. The heuristic was considerably faster, completing ten runs in only 2.514 seconds compared with the ILP approach's 301.533 seconds.

The results show that both approaches have advantages. The ILP model is useful when finding a shorter route and obtaining evidence about optimality are priorities. The heuristic approach is more practical when computational time is limited, and a good approximate solution is sufficient.

Overall, the study demonstrates that optimization algorithms can be used to improve PCB drill-head routing and highlights the trade-off between route quality and computational cost.

9.3 Future work

Future experiments could use more independently selected subsets and a larger number of repeated runs to better understand the consistency of the results. The ILP and heuristic approaches could also be compared using equal execution-time limits.

The heuristic could be improved by investigating more advanced techniques such as 3-opt and iterated local search. These methods may help overcome some of the limitations of 2-opt and produce routes closer to the global optimum.

Further improvements to the ILP approach could include stronger subtour elimination techniques to help the solver improve its lower bound more efficiently.

Finally, the model could be extended to represent an actual PCB drilling machine by including travel speed, acceleration, drilling time, tool changes, and machine-specific starting and finishing positions. This would allow the optimization results to be evaluated using actual manufacturing time rather than distance alone.

References

[1] G. Reinelt, “pcb442,” TSPLIB, original dataset. https://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/tsp/pcb442.tsp.gz.

[2] G. Reinelt, TSPLIB 95, 1995, Section 2.1. https://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/tsp/tsp95.pdf.

[3] G. Reinelt, “Optimal solutions for symmetric TSPs,” TSPLIB. https://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/tsp/TSP-BEST.html.

[4] M. Held and R. M. Karp, “A dynamic programming approach to sequencing problems,” Journal of the Society for Industrial and Applied Mathematics, vol. 10, no. 1, pp. 196–210, 1962. https://doi.org/10.1137/0110015.

[5] G. A. Croes, “A method for solving traveling-salesman problems,” Operations Research, vol. 6, no. 6, pp. 791–812, 1958. https://doi.org/10.1287/opre.6.6.791.

[6] SCIP Optimization Suite, PySCIPOpt documentation and TSP constraint-handler example. https://pyscipopt.readthedocs.io/ and https://github.com/scipopt/PySCIPOpt/blob/master/tests/test_tsp.py.

Appendix A: Complete Source Code

A.1 scripts/exact_method.py

```
"""Held–Karp dynamic programming for small symmetric TSP instances."""

import numpy as np


def held_karp(distance):
    """Return a globally optimal permutation and its closed-tour cost.

    State (mask, j) is a shortest path from node 0 through exactly mask,
    ending at j + 1. Bits represent nodes 1 through n - 1.
    Time O(n^2 2^n); space O(n 2^n). Limit n to protect memory/runtime.
    """
    n = len(distance)
    if not 3 <= n <= 20:
        raise ValueError("Held–Karp supports 3 to 20 nodes in this project")
    m = n - 1
    states = 1 << m
    infinity = np.iinfo(np.int64).max // 4
    cost = np.full((states, m), infinity, dtype=np.int64)
    parent = np.full((states, m), -1, dtype=np.int8)
    for j in range(m):
        cost[1 << j, j] = distance[0, j + 1]
    for mask in range(1, states):
        remaining = mask
        while remaining:
            bit = remaining & -remaining
            j = bit.bit_length() - 1
            previous = mask ^ bit
            if previous:
                candidates = previous
                best, predecessor = infinity, -1
                while candidates:
                    kbit = candidates & -candidates
                    k = kbit.bit_length() - 1
                    value = cost[previous, k] + distance[k + 1, j + 1]
                    if value < best:
                        best, predecessor = value, k
                    candidates ^= kbit
                cost[mask, j] = best
                parent[mask, j] = predecessor
```

```
            remaining ^= bit
    last_costs = cost[-1] + distance[1:, 0]
    j = int(np.argmin(last_costs))
    optimum = int(last_costs[j])
    mask, reverse_path = states - 1, []
    while mask:
        reverse_path.append(j + 1)
        predecessor = int(parent[mask, j])
        mask ^= 1 << j
        j = predecessor
    return np.array([0] + reverse_path[::-1]), optimum
```

A.2 scripts/heuristic_method.py

```
"""Random-start nearest neighbour followed by best-improvement 2-opt."""

import numpy as np


def tour_length(route, distance):
    """Include the last-to-first edge; route itself contains no repeat."""
    route = np.asarray(route, dtype=int)
    return int(distance[route, np.roll(route, -1)].sum())


def local_search(distance, seed):
    """Return a feasible 2-opt local optimum, initial cost and cost history.

    Only the start is random. Nearest-neighbour ties use smallest index;
    equal 2-opt improvements use row-major order. No iteration/time cap.
    """
    n = len(distance)
    rng = np.random.default_rng(seed)
    route = [int(rng.integers(n))]
    unused = set(range(n)) - set(route)
    while unused:
        node = min(unused, key=lambda j: (distance[route[-1], j], j))
        route.append(node)
        unused.remove(node)
    route = np.array(route)
    history = [tour_length(route, distance)]
    # Nonadjacent edge pairs only; first and last edges are adjacent too.
    i, j = np.triu_indices(n, k=2)
    valid = ~((i == 0) & (j == n - 1))
    i, j = i[valid], j[valid]
```

```
    while len(i):
        successor = np.roll(route, -1)
        delta = (distance[route[i], route[j]]
                 + distance[successor[i], successor[j]]
                 - distance[route[i], successor[i]]
                 - distance[route[j], successor[j]])
        move = int(np.argmin(delta))
        if delta[move] >= 0:
            break
        left, right = int(i[move]) + 1, int(j[move]) + 1
        route[left:right] = route[left:right][::-1]
        history.append(history[-1] + int(delta[move]))
    assert tour_length(route, distance) == history[-1]
    return route, history[0], history
```

A.3 scripts/main.py

```
"""Reproduce the pcb442 experiments and figures from the local dataset."""

import csv
import hashlib
import json
import platform
import sys
from datetime import datetime, timezone
from pathlib import Path
from time import perf_counter

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

from exact_method import held_karp
from heuristic_method import local_search, tour_length

ROOT = Path(__file__).resolve().parents[1]
SIZES = (8, 10, 12, 14, 16, 18, 50, 100, 200, 442)
SEEDS = tuple(range(10))
SUBSET_SEED = 2026
EXACT_REPEATS = 3
FULL_OPTIMUM = 50778  # Published by TSPLIB; not computed by our exact solver.


def load_dataset(path):
```

```
    """Read and validate the original TSPLIB coordinate file."""
    metadata, rows = {}, []
    coordinates = False
    for line in path.read_text().splitlines():
        line = line.strip()
        if line == "NODE_COORD_SECTION":
            coordinates = True
        elif line == "EOF":
            break
        elif coordinates and line:
            node, x, y = line.split()
            rows.append((int(node), float(x), float(y)))
        elif ":" in line:
            key, value = line.split(":", 1)
            metadata[key.strip()] = value.strip()
    if (metadata.get("NAME") != "pcb442"
            or metadata.get("TYPE") != "TSP"
            or metadata.get("EDGE_WEIGHT_TYPE") != "EUC_2D"
            or int(metadata.get("DIMENSION", 0)) != 442
            or len(rows) != 442):
        raise ValueError("Expected original pcb442 EUC_2D data")
    if [row[0] for row in rows] != list(range(1, 443)):
        raise ValueError("Expected unique consecutive TSPLIB node IDs")
    xy = np.array([row[1:] for row in rows])
    if not np.isfinite(xy).all():
        raise ValueError("Coordinates must be finite")
    return metadata, xy


def distance_matrix(xy):
        """TSPLIB EUC_2D uses nearest integer, with halves rounded upward."""
    difference = xy[:, None, :] - xy[None, :, :]
    return np.floor(np.sqrt((difference ** 2).sum(axis=2)) + 0.5
                    ).astype(np.int64)


def validate_tour(route, distance, expected):
    if sorted(route) != list(range(len(distance))):
        raise ValueError("Tour must visit every node exactly once")
    if tour_length(route, distance) != expected:
        raise ValueError("Closed-tour cost does not match")


def write_csv(path, rows):
```

```
    with path.open("w", newline="") as file:
        writer = csv.DictWriter(file, fieldnames=list(rows[0]))
        writer.writeheader()
        writer.writerows(rows)


def summarize(rows):
    summary = []
    for n in SIZES:
        for method in ("held_karp", "two_opt"):
            group = [r for r in rows if r["n"] == n
                     and r["method"] == method]
            if not group:
                continue
            costs = [r["distance"] for r in group]
            times = [r["runtime_seconds"] for r in group]
            reference = group[0]["reference_distance"]
            summary.append(dict(
                n=n, method=method, runs=len(group), best=min(costs),
                mean=float(np.mean(costs)), worst=max(costs),
                sd=float(np.std(costs, ddof=1)),
                mean_seconds=float(np.mean(times)),
                sd_seconds=float(np.std(times, ddof=1)),
                reference_distance=reference,
                mean_gap_percent=(100 * (np.mean(costs) / reference - 1)
                                  if reference else ""),
                feasible_runs=sum(r["feasible"] for r in group)))
    return summary


def figures(xy, rows, summary, best_route, history, output):
    plt.rcParams.update({"font.size": 10, "axes.spines.top": False,
                         "axes.spines.right": False})

    def save(name):
        plt.savefig(output / name, dpi=180, bbox_inches="tight")
        plt.close()

    for name, title, route in (
            ("drilling_locations.png", "pcb442 drilling locations", None),
            ("optimized_route.png", "Best 2-opt closed tour", best_route)):
        fig, ax = plt.subplots(figsize=(7, 6))
        if route is not None:
            closed = np.append(route, route[0])
```

```
            ax.plot(xy[closed, 0], xy[closed, 1], lw=0.8, color="#28638a")
        ax.scatter(*xy[route[0]], s=65, marker="s", color="#c44b28",
                   label="Start / return", zorder=3)
        ax.legend()
    ax.scatter(xy[:, 0], xy[:, 1], s=6, color="#172e40")
    ax.set(title=title, xlabel="x (dataset units)",
           ylabel="y (dataset units)", aspect="equal")
    save(name)
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
for method, label in (("held_karp", "Held–Karp"),
                      ("two_opt", "NN + 2-opt")):
    group = [r for r in summary if r["method"] == method]
    axes[0].plot([r["n"] for r in group],
                 [r["mean_seconds"] for r in group], "o-", label=label)
axes[0].set(xscale="log", yscale="log", xlabel="Locations (log scale)",
            ylabel="Mean solve time (s, log scale)")
axes[0].legend()
small = [r for r in summary if r["method"] == "two_opt" and r["n"] <= 18]
axes[1].bar([str(r["n"]) for r in small],
            [r["mean_gap_percent"] for r in small], color="#28638a")
axes[1].set(xlabel="Locations", ylabel="Mean gap to exact optimum (%)")
fig.tight_layout()
save("algorithm_comparison.png")
full = [r for r in rows if r["n"] == 442]
fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(SEEDS, [r["initial_distance"] for r in full], "o-", label="Initial NN")
ax.plot(SEEDS, [r["distance"] for r in full], "o-", label="After 2-opt")
ax.axhline(FULL_OPTIMUM, color="#c44b28", ls="--", label="Published optimum")
ax.set(xlabel="Random seed", ylabel="Closed-tour distance", xticks=SEEDS)
ax.legend()
save("full_benchmark_runs.png")
fig, ax = plt.subplots(figsize=(8, 3.5))
ax.plot(range(len(history)), history, color="#28638a")
ax.axhline(FULL_OPTIMUM, color="#c44b28", ls="--", label="Published optimum")
ax.set(xlabel="Accepted 2-opt moves (best full run)",
       ylabel="Closed-tour distance")
ax.legend()
save("convergence.png")


def main():
    output = ROOT / "output"
    output.mkdir(exist_ok=True)
    dataset = ROOT / "dataset" / "pcb442.tsp"
```

```
metadata, xy = load_dataset(dataset)
full_distance = distance_matrix(xy)
order = np.random.default_rng(SUBSET_SEED).permutation(len(xy))
rows, routes, subsets = [], {}, {}
best_full, best_history, best_cost = None, None, float("inf")
started = datetime.now(timezone.utc).isoformat()
for n in SIZES:
    # Nested subsets; full instance retains the original TSPLIB order.
    indices = np.sort(order[:n]) if n < 442 else np.arange(442)
    subsets[str(n)] = (indices + 1).tolist()
    d = full_distance[np.ix_(indices, indices)]
    reference = FULL_OPTIMUM if n == 442 else None
    jobs = ([('held_karp', run) for run in range(EXACT_REPEATS)]
            if n <= 18 else [])
        jobs += [('two_opt', seed) for seed in SEEDS]
    for method, run in jobs:
        start = perf_counter()
        if method == "held_karp":
            route, cost = held_karp(d)
            initial, history = None, []
        else:
            route, initial, history = local_search(d, run)
            cost = history[-1]
        elapsed = perf_counter() - start
        validate_tour(route, d, cost)
        if method == "held_karp":
            reference = cost
        if reference is not None and cost < reference:
            raise ValueError("Cost is below verified reference")
        key = f"{n}_{method}_{run}"
        routes[key] = (indices[route] + 1).tolist()
        rows.append(dict(
            n=n, method=method, run=run,
            seed=run if method == "two_opt" else "",
            distance=cost, initial_distance=initial,
            runtime_seconds=elapsed, feasible=True,
            status="optimal" if method == "held_karp" else "local_optimum",
            reference_distance=reference,
            gap_percent=100 * (cost / reference - 1) if reference else "",
            accepted_moves=len(history) - 1 if history else ""))
        if n == 442 and cost < best_cost:
            best_full, best_history, best_cost = route.copy(), history, cost
    print(f"Completed n={n}", flush=True)
summary = summarize(rows)
```

```
    write_csv(output / "results.csv", rows)
    write_csv(output / "summary.csv", summary)
    write_csv(output / "convergence.csv",
              [dict(move=i, distance=c) for i, c in enumerate(best_history)])
    (output / "routes.json").write_text(json.dumps(routes, indent=2) + "\n")
    details = dict(
        started_utc=started, python=sys.version, platform=platform.platform(),
        machine=platform.machine(), numpy=np.__version__,
        matplotlib=matplotlib.__version__, dataset=metadata,
        dataset_sha256=hashlib.sha256(dataset.read_bytes()).hexdigest(),
        source_sha256={p.name: hashlib.sha256(p.read_bytes()).hexdigest()
                       for p in sorted((ROOT / "scripts").glob("*.py"))},
        subset_seed=SUBSET_SEED, subset_node_ids=subsets, seeds=SEEDS,
        exact_repeats=EXACT_REPEATS, exact_limit=18,
        heuristic="random-start NN, best-improvement 2-opt until no gain",
        tie_breaking="smallest node index / first row-major edge pair",
        timing="perf_counter; solver only, including initialization; serial",
        published_full_optimum=FULL_OPTIMUM,
        reference_url="https://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/"
                      "tsp/TSP-BEST.html")
    (output / "experiment_details.json").write_text(
        json.dumps(details, indent=2) + "\n")
    figures(xy, rows, summary, best_full, best_history, output)
    print(f"Best full tour: {best_cost}; all {len(rows)} tours feasible")


if __name__ == "__main__":
    main()
```

A.4 scripts/verify.py

```
"""Independent correctness checks; run before the experiment pipeline."""

from itertools import permutations

import numpy as np

from exact_method import held_karp
from heuristic_method import local_search, tour_length
from main import ROOT, distance_matrix, load_dataset, validate_tour


def main():
    _, xy = load_dataset(ROOT / "dataset" / "pcb442.tsp")
    assert xy.shape == (442, 2)
```

```
    distance = distance_matrix(xy)
    assert np.array_equal(distance, distance.T)
    assert np.all(np.diag(distance) == 0)
    # Half-up rounding must differ from Python's banker's rounding at 2.5.
    assert distance_matrix(np.array([[0, 0], [2.5, 0]]))[0, 1] == 3
    rng = np.random.default_rng(123)
    for n in (4, 6, 8):
        d = distance_matrix(rng.uniform(0, 100, (n, 2)))
        brute = min(tour_length((0,) + p, d)
                    for p in permutations(range(1, n)))
        route, optimum = held_karp(d)
        validate_tour(route, d, optimum)
        assert optimum == brute
        for seed in range(5):
            route, initial, history = local_search(d, seed)
            validate_tour(route, d, history[-1])
            assert initial >= history[-1] >= optimum
            assert all(a > b for a, b in zip(history, history[1:]))
            # Exhaustively verify termination with no improving 2-opt move.
            for i in range(n):
                for j in range(i + 2, n):
                    trial = route.copy()
                    trial[i + 1:j + 1] = trial[i + 1:j + 1][::-1]
                    assert tour_length(trial, d) >= history[-1]
    try:
        validate_tour([0, 1, 1, 3], np.zeros((4, 4)), 0)
    except ValueError:
        pass
    else:
        raise AssertionError("Duplicate visit was not rejected")
    print("PASS: rounding, closure, brute-force agreement, feasibility, 2-opt")


if __name__ == "__main__":
    main()
```

A.5 scripts/ilp_method.py

```
"""Exact TSP model with SCIP branch-and-bound and subtour constraints.

Connectivity callbacks follow the PySCIPOpt constraint-handler interface:
https://github.com/scipopt/PySCIPOpt/blob/master/tests/test_tsp.py
"""

from time import perf_counter
```

```
import numpy as np
from pyscipopt import Conshdlr, Model, SCIP_RESULT, quicksum

from heuristic_method import local_search, tour_length


class Connectivity(Conshdlr):
    """Reject disconnected cycles and add valid subtour inequalities."""

    def __init__(self, n, edges):
        self.n, self.edges, self.cuts = n, edges, 0

    def components(self, solution=None):
        neighbours = [[] for _ in range(self.n)]
        for (a, b), variable in self.edges.items():
            if self.model.getSolVal(solution, variable) > 1e-6:
                neighbours[a].append(b)
                neighbours[b].append(a)
        remaining, groups = set(range(self.n)), []
        while remaining:
            stack = [remaining.pop()]
            group = set(stack)
            while stack:
                for b in neighbours[stack.pop()]:
                    if b in remaining:
                        remaining.remove(b)
                        group.add(b)
                        stack.append(b)
            groups.append(sorted(group))
        return groups

    def conscheck(self, constraints, solution, *args, **kwargs):
        connected = len(self.components(solution)) == 1
            return {"result": (SCIP_RESULT.FEASIBLE if connected
                        else SCIP_RESULT.INFEASIBLE)}

    def consenfolp(self, constraints, *args, **kwargs):
        groups = self.components()
        if len(groups) == 1:
            return {"result": SCIP_RESULT.FEASIBLE}
        for group in groups:
            internal = (self.edges[a, b] for k, a in enumerate(group)
                        for b in group[k + 1:])
```

```
            self.model.addCons(quicksum(internal) <= len(group) - 1)
            self.cuts += 1
        return {"result": SCIP_RESULT.CONSADDED}

    def conslock(self, *args):
        # Degree equalities already lock every variable in both directions.
        pass


def solve_ilp(distance, time_limit=300):
    """Return a valid tour and solver evidence; a time limit is not proof.

    All n(n-1)/2 edges are included. Seed-0 NN + 2-opt provides a feasible
    warm start, included in total runtime. No published tour or value is
    supplied to the optimizer. Disconnected cycle covers are never returned.
    """
    start = perf_counter()
    n = len(distance)
    model = Model("pcb_tsp")
    model.hideOutput()
    model.setIntParam("parallel/maxnthreads", 1)
    model.setIntParam("randomization/randomseedshift", 0)
    model.setBoolParam("misc/allowstrongdualreds", False)
    model.setRealParam("limits/time", float(time_limit))
    model.setRealParam("limits/gap", 0.0)
    model.setRealParam("numerics/feastol", 1e-6)
    edges = {(a, b): model.addVar(vtype="B", obj=int(distance[a, b]))
             for a in range(n) for b in range(a + 1, n)}
    incident = [[] for _ in range(n)]
    for (a, b), variable in edges.items():
        incident[a].append(variable)
        incident[b].append(variable)
    for variables in incident:
        model.addCons(quicksum(variables) == 2)
    handler = Connectivity(n, edges)
    model.includeConshdlr(handler, "connectivity", "one complete tour",
                          chckpriority=-10, needscons=False)
    initial, _, _ = local_search(distance, seed=0)
    solution = model.createSol()
    for a, b in zip(initial, np.roll(initial, -1)):
        model.setSolVal(solution, edges[min(a, b), max(a, b)], 1)
    model.addSol(solution)
    model.optimize()
    neighbours = [[] for _ in range(n)]
```

```
    for (a, b), variable in edges.items():
        if model.getVal(variable) > 0.5:
            neighbours[a].append(b)
            neighbours[b].append(a)
    if any(len(v) != 2 for v in neighbours):
        raise ValueError("SCIP did not return degree-two edges")
    route, previous, current = [], -1, 0
    while current not in route:
        route.append(current)
        following = next(v for v in neighbours[current] if v != previous)
        previous, current = current, following
    if current != 0 or sorted(route) != list(range(n)):
        raise ValueError("SCIP returned a disconnected or invalid cycle")
    cost = tour_length(route, distance)
    if abs(cost - model.getObjVal()) > 1e-5:
        raise ValueError("SCIP objective differs from integer tour cost")
    evidence = dict(
        distance=cost, initial_distance=tour_length(initial, distance),
        runtime_seconds=perf_counter() - start,
        solver_seconds=model.getSolvingTime(),
        status=str(model.getStatus()), lower_bound=model.getDualbound(),
        solver_gap=model.getGap(), branch_nodes=model.getNNodes(),
        subtour_constraints=handler.cuts, time_limit_seconds=time_limit,
        variables=len(edges), scip_version="{}.{}.{}".format(
            model.getMajorVersion(), model.getMinorVersion(),
            model.getTechVersion()))
    return np.array(route), evidence
```

A.6 scripts/compare_models.py

```
"""Run the full-board ILP comparison and plot saved complete tours."""

import argparse
import csv
import hashlib
import json
import platform
from time import perf_counter

import matplotlib.pyplot as plt
import numpy as np
import pyscipopt

from exact_method import held_karp
from heuristic_method import local_search
```

```
from ilp_method import solve_ilp
from main import (FULL_OPTIMUM, ROOT, SEEDS, distance_matrix, load_dataset,
                  validate_tour, write_csv)


def draw_pair(xy, routes, titles, path):
    """Draw every location and edge on identical axes; arrows show direction."""
    fig, axes = plt.subplots(1, 2, figsize=(11, 6.7), sharex=True, sharey=True)
    for ax, route, title in zip(axes, routes, titles):
        route = np.asarray(route)
        route = np.roll(route, -int(np.where(route == 0)[0][0]))
        closed = np.append(route, route[0])
        points = xy[closed]
        ax.plot(points[:, 0], points[:, 1], color="#28638a", lw=0.75)
        ax.scatter(xy[:, 0], xy[:, 1], s=8, color="#172e40", zorder=3)
        ax.scatter(*xy[0], marker="s", s=40, color="#c44b28",
                   label="Common start / return", zorder=4)
        for k in range(0, len(route), max(1, len(route) // 12)):
            a, b = points[k], points[k + 1]
            ax.annotate("", xy=b, xytext=a,
                        arrowprops=dict(arrowstyle="->", color="#c44b28",
                                        lw=0.9, mutation_scale=9))
        ax.set(title=title, xlabel="x (dataset units)", aspect="equal")
        ax.legend(loc="upper left", fontsize=8)
    axes[0].set_ylabel("y (dataset units)")
    fig.tight_layout()
    fig.savefig(path, dpi=220, bbox_inches="tight")
    plt.close(fig)


def edge_set(route):
    return {tuple(sorted((int(a), int(b))))
            for a, b in zip(route, np.roll(route, -1))}


def export_comparison(record, xy, distance, output):
    runs = record["runs"]
    exact = runs[0]
    heuristic = min(runs[1:], key=lambda r: r["distance"])
    best = [exact, heuristic]
    routes = [np.array(r["node_ids"]) - 1 for r in best]
    for route, result in zip(routes, best):
        validate_tour(route, distance, result["distance"])
    common = len(edge_set(routes[0]) & edge_set(routes[1]))
```

```
summary = []
for route, result, group in zip(routes, best, [runs[:1], runs[1:]]):
    lengths = distance[route, np.roll(route, -1)]
    summary.append(dict(
        method=result["method"], runs=len(group),
        best_distance=result["distance"],
        mean_distance=float(np.mean([r["distance"] for r in group])),
        best_run_seconds=result["runtime_seconds"],
        mean_seconds=float(np.mean([r["runtime_seconds"] for r in group])),
        total_seconds=sum(r["runtime_seconds"] for r in group),
        best_seed=result.get("seed", ""), status=result["status"],
        lower_bound=result.get("lower_bound", ""),
        solver_gap=result.get("solver_gap", ""),
        reference_gap_percent=100 * (result["distance"] / FULL_OPTIMUM - 1),
        visited_nodes=len(set(route)), edges=len(route),
        mean_edge=float(np.mean(lengths)), longest_edge=int(max(lengths)),
        shared_edges=common))
write_csv(output / "best_model_comparison.csv", summary)
status = "proven optimal" if exact["status"] == "optimal" else "not proven optimal"
titles = [f"Exact ILP model ({status})\nDistance {exact['distance']:,}",
          f"Best NN + 2-opt (seed {heuristic['seed']})\n"
          f"Distance {heuristic['distance']:,}"]
draw_pair(xy, routes, titles, output / "complete_routes_comparison.png")
old = list(csv.DictReader((output / "results.csv").open()))
saved = json.loads((output / "routes.json").read_text())
subsets = json.loads((output / "experiment_details.json").read_text())
small = []
for n in (8, 10, 12, 14, 16, 18):
    group = [r for r in old if int(r["n"]) == n]
    e = next(r for r in group if r["method"] == "held_karp")
    hs = [r for r in group if r["method"] == "two_opt"]
    exact_times = [float(r["runtime_seconds"]) for r in group
                   if r["method"] == "held_karp"]
    b = min(hs, key=lambda r: int(r["distance"]))
    small.append(dict(
        n=n, exact_distance=int(e["distance"]),
        heuristic_best=int(b["distance"]),
        excess=int(b["distance"]) - int(e["distance"]),
        best_gap_percent=100 * (int(b["distance"]) / int(e["distance"]) - 1),
        exact_mean_seconds=float(np.mean(exact_times)),
        heuristic_total_seconds=sum(float(r["runtime_seconds"]) for r in hs)))
    if n == 10:
        ids = subsets["subset_node_ids"]["10"]
        keys = [f"10_{r['method']}_{r['run']}" for r in [e, b]]
```

```
        small_routes = [[ids.index(v) for v in saved[k]] for k in keys]
        draw_pair(xy[np.array(ids) - 1], small_routes,
                  [f"Held–Karp optimum: {e['distance']}",
                   f"Best NN + 2-opt: {b['distance']}"],
                  output / "subset_routes_comparison.png")
    write_csv(output / "best_subset_comparison.csv", small)


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--plots-only", action="store_true")
        args = parser.parse_args()
    output = ROOT / "output"
    _, xy = load_dataset(ROOT / "dataset" / "pcb442.tsp")
    distance = distance_matrix(xy)
    path = output / "full_comparison.json"
    if args.plots_only:
        record = json.loads(path.read_text())
    else:
        # Cross-check the ILP on independent small cases, including subtours.
        for n in (4, 6, 8, 10):
            sample = np.random.default_rng(n).uniform(0, 100, (n, 2))
            d = distance_matrix(sample)
            route, evidence = solve_ilp(d, time_limit=30)
            _, optimum = held_karp(d)
            validate_tour(route, d, optimum)
            assert evidence["status"] == "optimal"
        d = np.full((6, 6), 10, dtype=int)
        d[:3, :3] = d[3:, 3:] = 1
        np.fill_diagonal(d, 0)
        route, evidence = solve_ilp(d, time_limit=30)
        validate_tour(route, d, 24)
        assert evidence["status"] == "optimal"
        print("ILP checks passed; solving all 442 nodes...", flush=True)
        route, evidence = solve_ilp(distance, time_limit=300)
        runs = [dict(method="ilp", **evidence, node_ids=(route + 1).tolist())]
        print(f"ILP: {evidence['distance']}, {evidence['status']}", flush=True)
        for seed in SEEDS:
            start = perf_counter()
            route, initial, history = local_search(distance, seed)
            elapsed = perf_counter() - start
            validate_tour(route, distance, history[-1])
            runs.append(dict(method="two_opt", seed=seed,
                             distance=history[-1], initial_distance=initial,
```

```
                    runtime_seconds=elapsed, status="local_optimum",
                    node_ids=(route + 1).tolist()))
        record = dict(
            runs=runs, platform=platform.platform(), python=platform.python_version(),
            numpy=np.__version__, pyscipopt=pyscipopt.__version__,
            source_sha256={p.name: hashlib.sha256(p.read_bytes()).hexdigest()
                           for p in sorted((ROOT / "scripts").glob("*.py"))},
            dataset_sha256=hashlib.sha256(
                (ROOT / "dataset/pcb442.tsp").read_bytes()).hexdigest(),
            settings=dict(threads=1, randomseedshift=0, relative_gap=0,
                          feasibility_tolerance=1e-6, strong_dual_reductions=False,
                          time_limit_seconds=300, warm_start="seed-0 NN + 2-opt"),
            timing="Serial; includes model, warm start, solve and extraction",
            verification="ILP agrees with Held–Karp on four independent cases; "
                         "two triangles test passed")
        path.write_text(json.dumps(record, indent=2) + "\n")
    export_comparison(record, xy, distance, output)
    print("Saved full-board and subset comparisons", flush=True)


if __name__ == "__main__":
    main()
```